

EVIDENCIA DE DESARROLLO: APP ESTACIONAMIENTO

POR JUSTO FRANCISCO LEÓN PASTOR BAAK

Descripción

Para el desarrollo de esta aplicación se utilizaron herramientas como Spring Boot y Maven, utilizando Hibernate con su proveedor JPA para la persistencia de datos, la estructura sigue el modelo de una arquitectura por capas con paquetes para dominio, dao, service y controller, esta aplicación permite administrar entradas y salidas de vehículos en un estacionamiento de cobro, para los vehículos de tipo residentes se genera un reporte con información mensual de su cobro reseteo de información (Se eliminan estancias de vehículos oficiales y se resetean los minutos acumulados de los residentes). Se construyó con base en APIs REST para facilitar la integración futura con una interfaz de usuario o cliente externo

1.Base de datos

Se utilizó una base de datos en Oracle con las siguientes entidades:

- **Vehículo:** placa, tipo de vehículo, minutos acumulados.
- **Estancia:** fecha de entrada, fecha de salida, relación con vehículo.

```
CREATE TABLE VEHICULO (  
    PLACA VARCHAR2(20) PRIMARY KEY,  
    TIPO_VEHICULO VARCHAR2(20) NOT NULL  
);
```

```
CREATE TABLE Estancia (  
    id NUMBER PRIMARY KEY,  
    hora_entrada TIMESTAMP NOT NULL,  
    hora_salida TIMESTAMP,  
    vehiculo_placa VARCHAR2(20) NOT NULL,  
    FOREIGN KEY (vehiculo_placa) REFERENCES Vehiculo(placa)  
);
```

```

CREATE TABLE VehiculoOficial (
    placa VARCHAR2(20) PRIMARY KEY,
    CONSTRAINT fk_vehiculo_oficial FOREIGN KEY (placa) REFERENCES Vehiculo(placa)
);

CREATE TABLE VehiculoResidente (
    placa VARCHAR2(20) PRIMARY KEY,
    minutos_acumulados NUMBER DEFAULT 0,
    CONSTRAINT fk_vehiculo_residente FOREIGN KEY (placa) REFERENCES Vehiculo(placa)
);

CREATE TABLE VehiculoNoResidente (
    placa VARCHAR2(20) PRIMARY KEY,
    CONSTRAINT fk_vehiculo_no_residente FOREIGN KEY (placa) REFERENCES Vehiculo(placa)
);

```

Se configuró application.properties con los parámetros para JPA

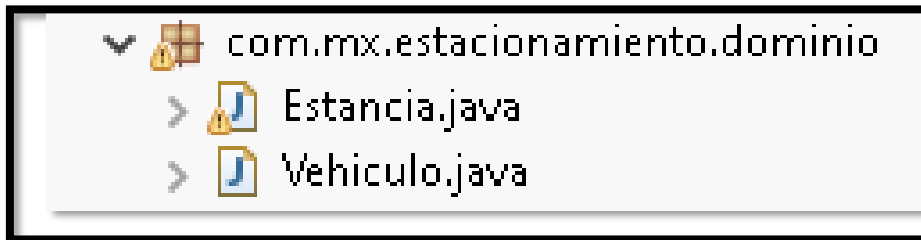
```

1 spring.application.name=ExamenFranciscoEstacionamiento
2 server.port = 9006
3
4
5
6 spring.datasource.url=jdbc:oracle:thin:@localhost:1521:xe
7 spring.datasource.username=System
8 spring.datasource.password=12345
9 spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
10
11
12 spring.jpa.properties.hibernate.format_sql = true
13
14 logging.level.org.hibernate.sql=debug
15 logging.level.org.hibernate.type.descriptor.sql.BasicBinder=trace

```

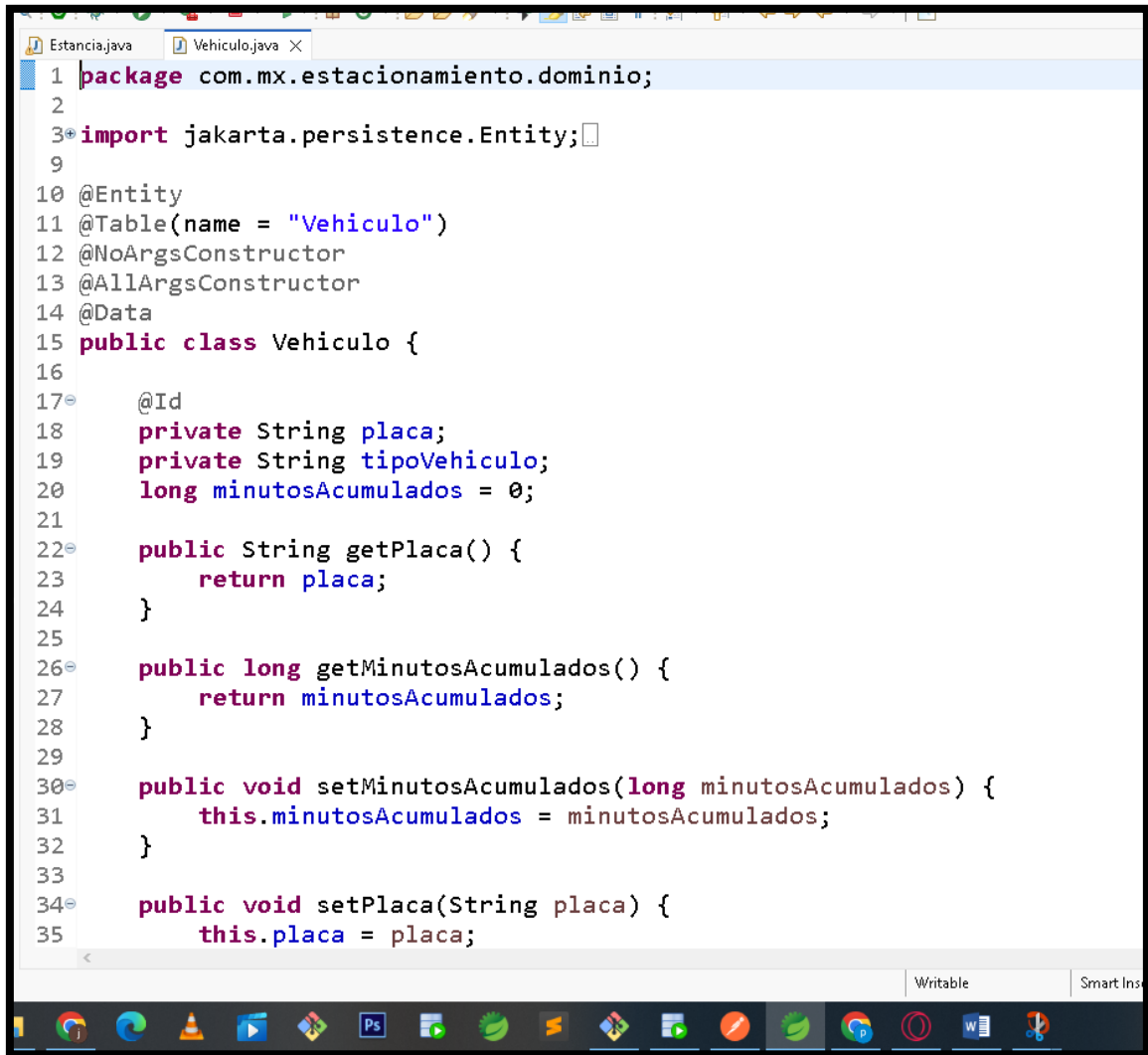
2.Dominio

Dentro del paquete com.mx.estacionamiento.dominio se definieron las entidades: Vehiculo.java y Estancia.java



Ambas están anotadas con @Entity y contienen las relaciones necesarias, getters/setters, y anotaciones de JPA(@Id, @ManyToOne)

```
CursoMarzo2025 - ExamenFranciscoEstacionamiento/src/main/java/com/mx/estacionamiento/dominio/Estancia.java - Spring Tool Suite 4
File Edit Source Refactor Navigate Search Project Run Window Help
Estancia.java X
1 package com.mx.estacionamiento.dominio;
2
3 import jakarta.persistence.Entity;
18
19 @Entity
20 @Table(name = "Estancia")
21 @NoArgsConstructor
22 @AllArgsConstructor
23 @Data
24 public class Estancia {
25
26     @Id
27     @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "estancia_seq")
28     @SequenceGenerator(name = "estancia_seq", sequenceName = "ESTANCIA_SEQ", allocationSize = 1)
29     private Long id;
30     @ManyToOne
31     private Vehiculo vehiculo;
32     @Column(name = "hora_entrada")
33     @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")
34     private Calendar horaEntrada;
35     @Column(name = "hora_salida")
36     @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")
37     private Calendar horaSalida;
38
39     public Long getId() {
40         return id;
41     }
42
43     public void setId(Long id) {
44         this.id = id;
45     }
46 }
```



```
1 package com.mx.estacionamiento.dominio;
2
3 import jakarta.persistence.Entity;
4
5
6
7
8
9
10 @Entity
11 @Table(name = "Vehiculo")
12 @NoArgsConstructor
13 @AllArgsConstructor
14 @Data
15 public class Vehiculo {
16
17     @Id
18     private String placa;
19     private String tipoVehiculo;
20     long minutosAcumulados = 0;
21
22     public String getPlaca() {
23         return placa;
24     }
25
26     public long getMinutosAcumulados() {
27         return minutosAcumulados;
28     }
29
30     public void setMinutosAcumulados(long minutosAcumulados) {
31         this.minutosAcumulados = minutosAcumulados;
32     }
33
34     public void setPlaca(String placa) {
35         this.placa = placa;
36     }
37 }
```

3.DAO

En el paquete com.mx.estacionamiento.dao se declararon los siguientes repositorios:

VehiculoRepository y EstanciasRepository ambos extienden JpaRepository y contienen personalizados como:

```

1 package com.mx.estacionamiento.dao;
2
3 import java.util.List;
4
5 @Repository
6 public interface EstanciaRepository extends JpaRepository<Estancia, Long> {
7
8     @Query("SELECT e FROM Estancia e WHERE e.vehiculo.placa = :placa AND e.horaSalida IS NULL OR")
9     List<Estancia> findEstanciasAbiertas(@Param("placa") String placa);
10
11     default Estancia findUltimaEstanciaAbierta(String placa) {
12         List<Estancia> estancias = findEstanciasAbiertas(placa);
13         return estancias.isEmpty() ? null : estancias.get(0); // Solo la más reciente
14     }
15
16     @Transactional
17     void deleteByVehiculo(Vehiculo vehiculo);
18
19     @Query("SELECT e FROM Estancia e WHERE e.vehiculo.tipoVehiculo = 'RESIDENTE'")
20     List<Estancia> findEstanciasResidentes();
21 }

```

```

1 package com.mx.estacionamiento.dao;
2
3 import java.util.List;
4
5 @Repository
6 public interface VehiculoRepository extends JpaRepository<Vehiculo, String> {
7
8     Vehiculo findByPlaca(String placa);
9
10     List<Vehiculo> findByTipoVehiculo(String tipoVehiculo);
11 }

```

4.Service

En com.mx.estacionamiento.service se creó la clase Implementacion.java donde se implementaron los casos de uso

- Registrar entrada
- Registrar salida
- Alta de vehículos
- Calcular cobros
- Generar informe de residentes
- Comenzar nuevo mes

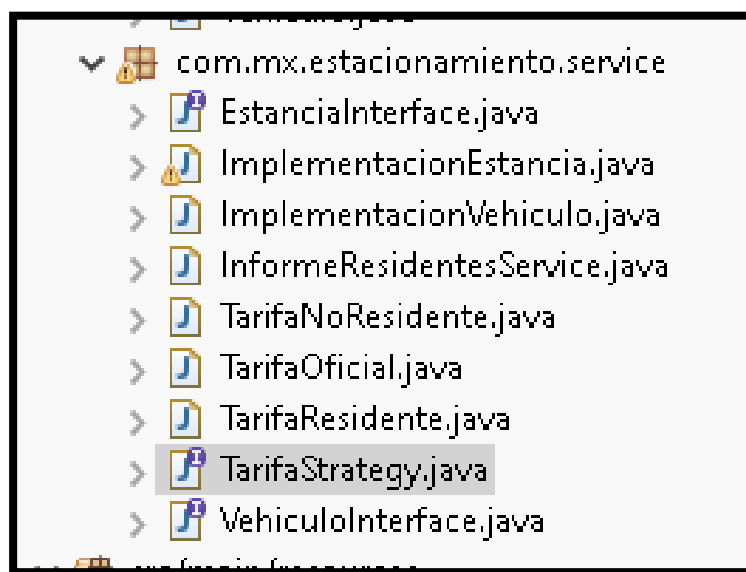
También se utilizó el patrón estrategia para los cobros:

```
Estancia.java  Vehiculo.java  EstanciaRepository.java  VehiculoRepository.java  ImplementacionEstancia.java X
42      return estanciaRepository.findById(id).orElse(null);
43  }
44
45  @Override
46  public List<Estancia> getAllEstancias() {
47      return estanciaRepository.findAll();
48  }
49
50  public double calcularCobro(Estancia estancia) {
51      if (estancia.getHoraEntrada() == null || estancia.getHoraSalida() == null) {
52          throw new IllegalArgumentException("Faltan datos de entrada o salida");
53      }
54
55      long minutos = calcularMinutos(estancia.getHoraEntrada(), estancia.getHoraSalida());
56      String tipoVehiculo = estancia.getVehiculo().getTipoVehiculo();
57
58      TarifaStrategy estrategia = obtenerEstrategia(tipoVehiculo);
59      return estrategia.calcularCobro(minutos);
60  }
61
62  private TarifaStrategy obtenerEstrategia(String tipoVehiculo) {
63      switch (tipoVehiculo.toUpperCase()) {
64          case "OFICIAL":
65              return new TarifaOficial();
66          case "RESIDENTE":
67              return new TarifaResidente();
68          case "NO_RESIDENTE":
69              return new TarifaNoResidente();
70          default:
71              throw new IllegalArgumentException("Tipo de vehículo no soportado: " + tipoVehiculo);
72      }
73  }
```

Writable Smart Insert 31:14:836

Ps [Taskbar icons] 5:57 AM 4/10/2025

```
1 package com.mx.estacionamiento.service;
2
3 public interface TarifaStrategy {
4     double calcularCobro(long minutos);
5 }
6
7
```



5.Controller

En el paquete controller se creó la clase Estancia, contiene endpoints REST para cada caso de uso

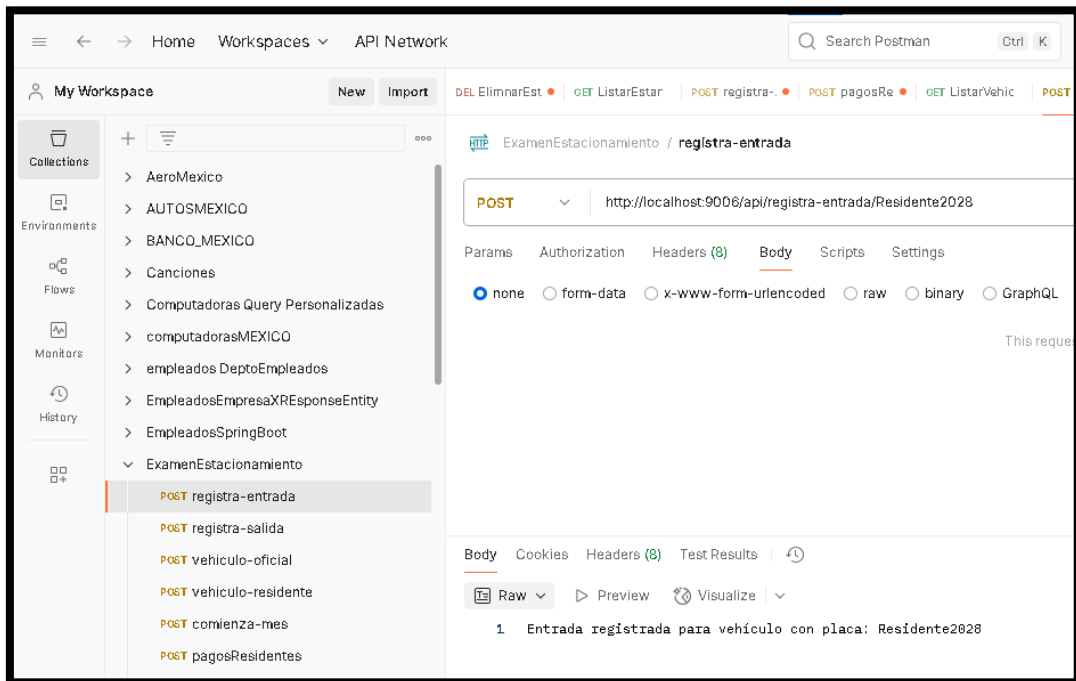
```
1 package com.mx.estacionamiento.controller;
2
3 import java.io.IOException;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 @RestController
27 @RequestMapping(path = "/api/")
28 @CrossOrigin
29 public class EstanciaController {
30
31     @Autowired
32     ImplementacionEstancia impEstancia;
33
34     // http://localhost:9006/api/estancias
35     @GetMapping(value = "estancias")
36     public ResponseEntity<> getAllEstancias() {
37         return ResponseEntity.status(HttpStatus.CREATED).body(impEstancia.getAllEstancias());
38     }
39
40     // http://localhost:9006/api/estancias
41     @DeleteMapping(value = "estancias/{id}")
42     public ResponseEntity<> deleteEstancias(@PathVariable("id") long id) {
43         impEstancia.eliminarEstancia(id);
44         return new ResponseEntity<String>("Delete:204", HttpStatus.OK);
45     }
46
47     // http://localhost:9006/EstanciaController/createEstancia
48     @PostMapping(value = "estancias")
49     public ResponseEntity<> createEstancia(@RequestBody Estancia estancia) {
50         impEstancia.createEstancia(estancia);
51         return new ResponseEntity<String>("Create:201", HttpStatus.OK);
52     }
53 }
```

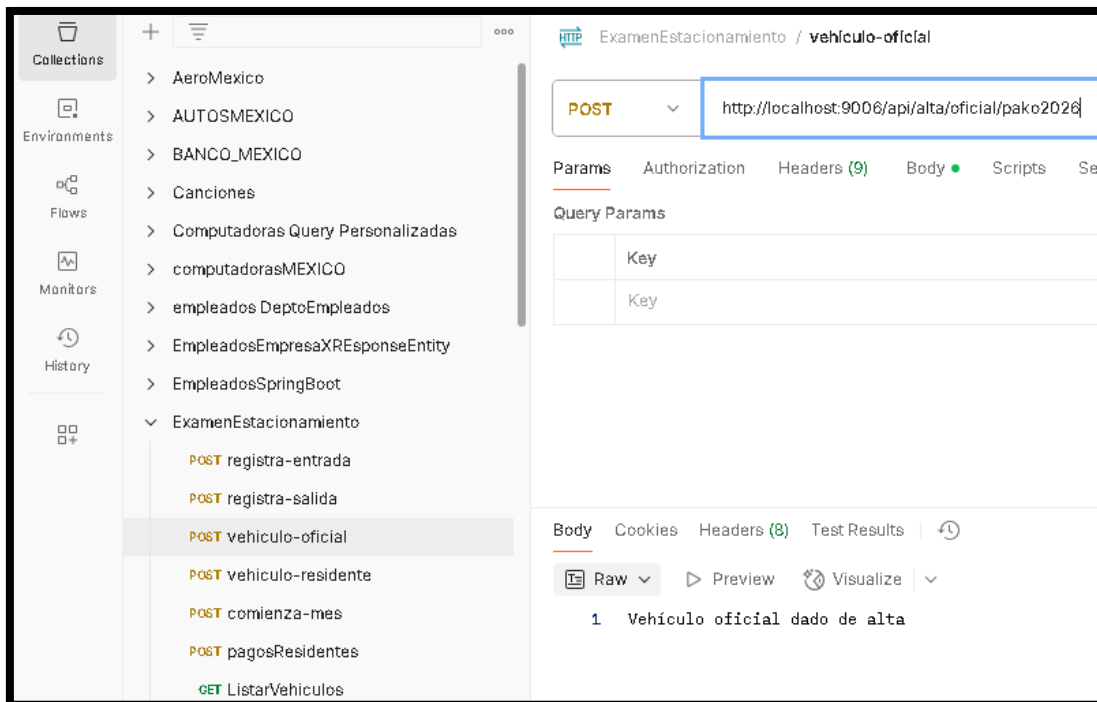
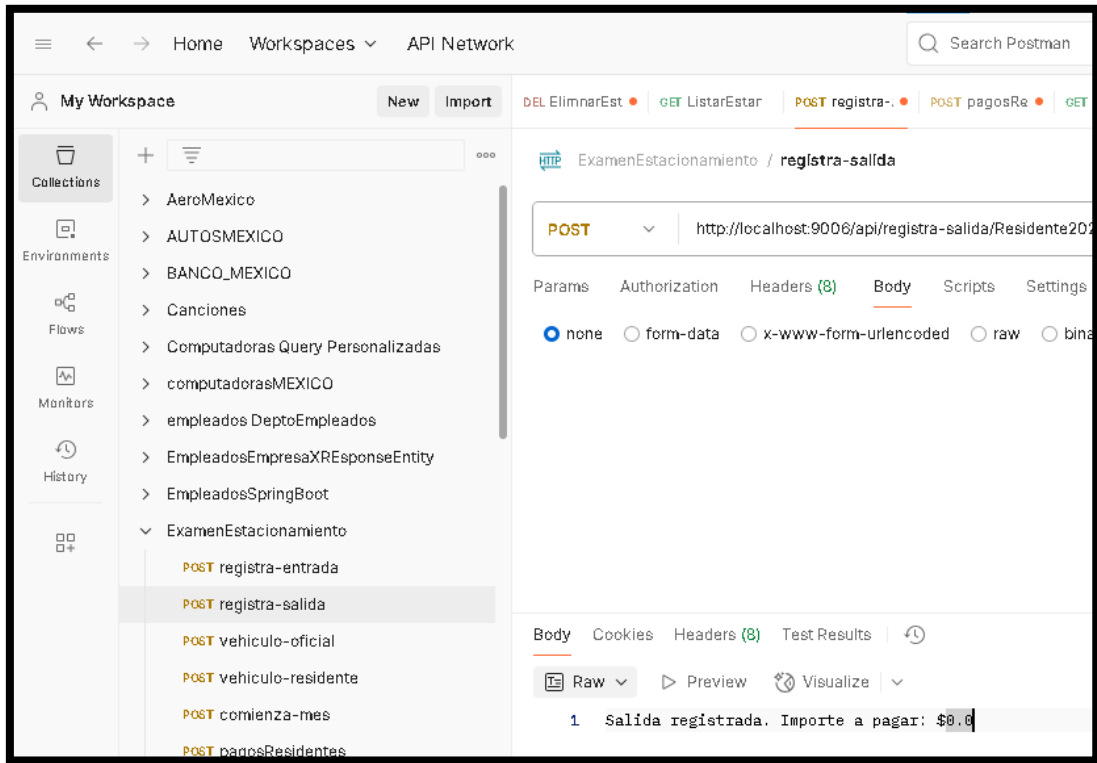
```
73
74 // http://localhost:9006/EstanciaController/comienza-mes/
75 @PostMapping("/comienza-mes")
76 public ResponseEntity<> comenzarMes() {
77     impEstancia.comenzarMes();
78     return ResponseEntity.ok("Mes reiniciado");
79 }
80
81 //http://localhost:9006/EstanciaController/pagos/residentes
82 @PostMapping("/pagos/residentes")
83 public ResponseEntity<> generarInforme(@RequestParam("nombreArchivo") String nombreArchivo)
84 {
85     try {
86         impEstancia.generarInformeResidentes(nombreArchivo);
87         return ResponseEntity.ok("Informe generado en: " + nombreArchivo);
88     } catch (IOException e) {
89         return ResponseEntity.status(500).body("Error al generar informe: " + e.getMessage());
90     }
91 }
92
93 @Autowired
94 InformeResidentesService informeResidentesService;
95
96 @PostMapping("/generar-informe-residentes")
97 public ResponseEntity<String> generarInformePagosResidentes(@RequestParam String nombreArch
98     informeResidentesService.generarInformePagosResidentes(nombreArchivo);
99     return ResponseEntity.ok("Informe generado correctamente: " + nombreArchivo);
100 }
101 }
102 }
```

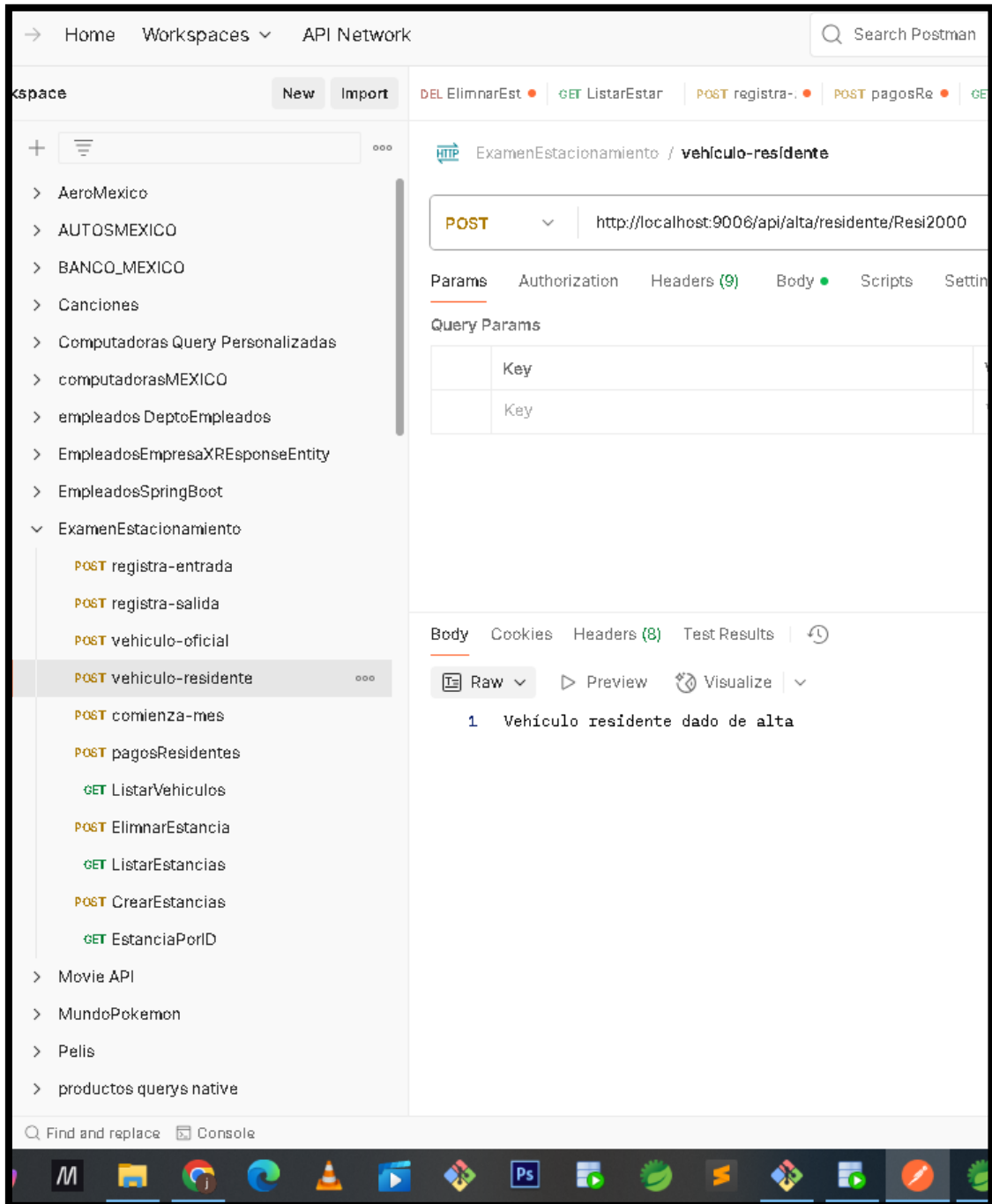

6. Pruebas de funcionalidad

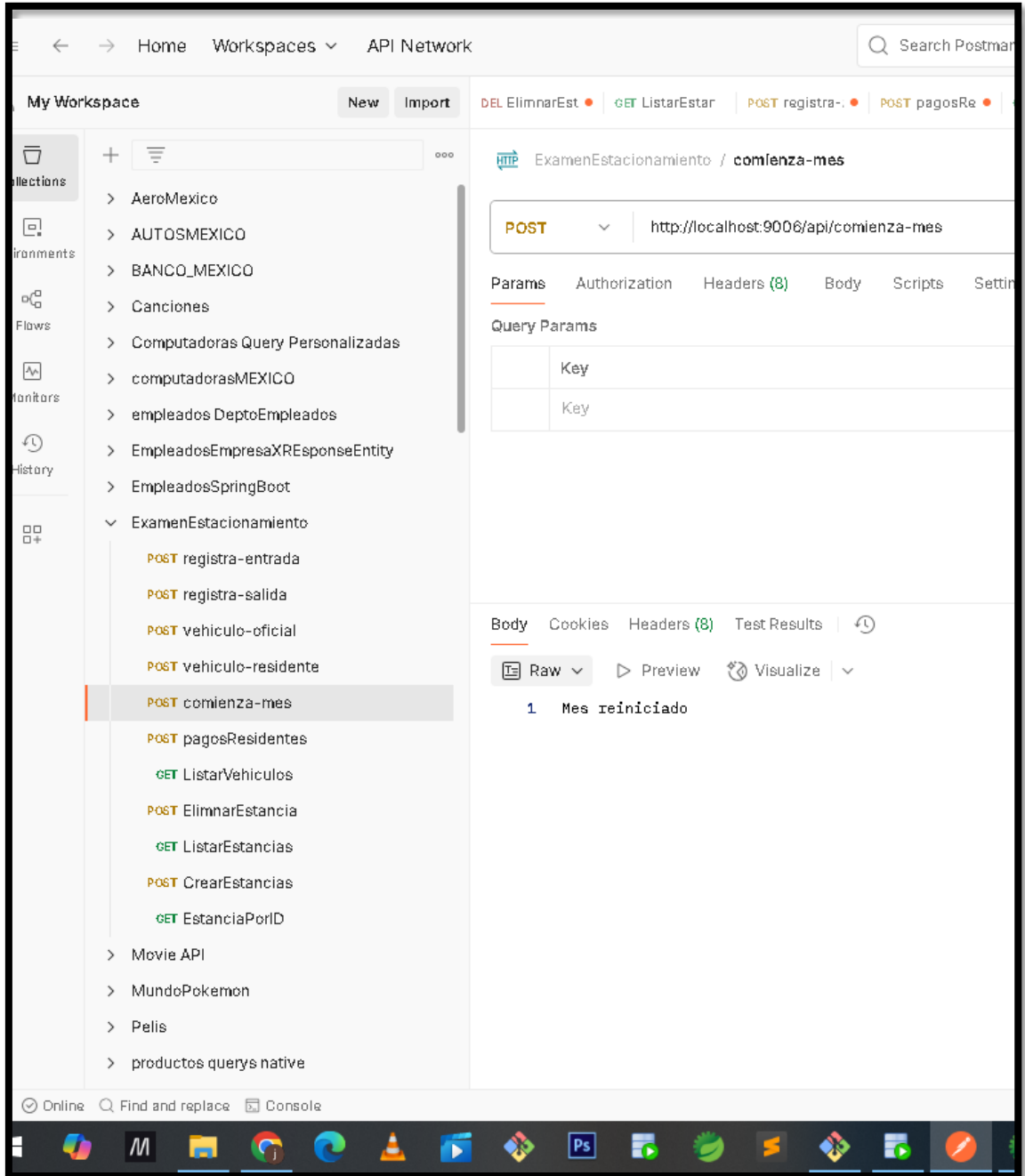
Las pruebas fueron realizadas en Postman y mediante logs . Se probaron los siguientes escenarios:

- Alta de vehículo
- Entrada de vehículo
- Salida de vehículo (cálculo correcto del tiempo)
- Comienzo de mes
- Generación del informe .txt de residentes con tiempo acumulado









ery Personalizadas

XICO

Empleados

saXResponseEntity

Boot

imiento

trada

lida

ficial

esidente

mes

identes

ulos

ancia

...

POST http://localhost:9006/api/pagos/residentes

Params Authorization Headers (9) Body Scripts Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	Key	Value
☒	nombreArchivo	informe_residentes.txt
	Key	Value

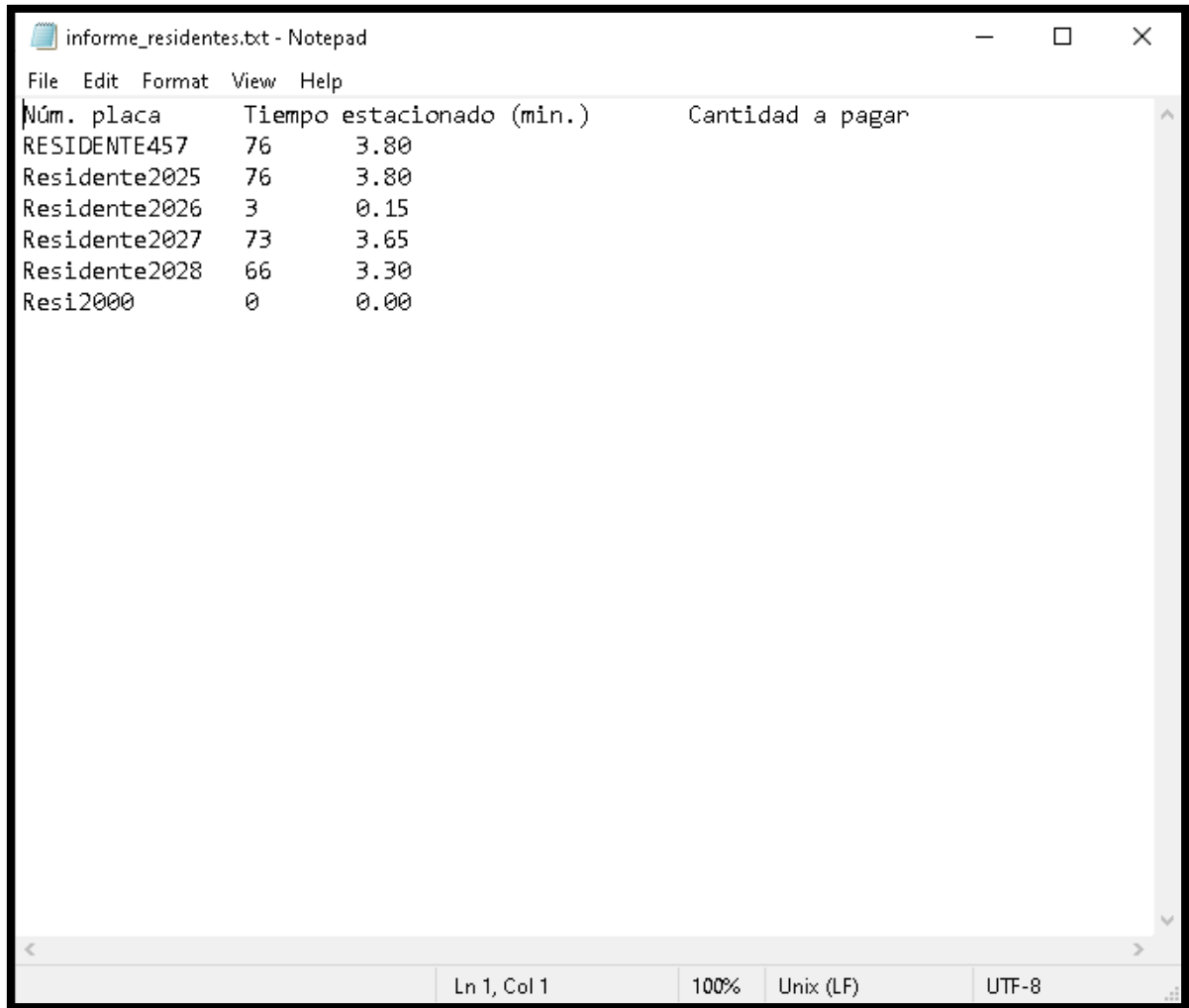
Body Cookies Headers (8) Test Results

Raw

Preview

Visualize

1 Informe generado en: informe_residentes.txt



Núm. placa	Tiempo estacionado (min.)	Cantidad a pagar
RESIDENTE457	76	3.80
Residente2025	76	3.80
Residente2026	3	0.15
Residente2027	73	3.65
Residente2028	66	3.30
Resi2000	0	0.00